BLEKINGE TEKNISKA HÖGSKOLA

BTH

Introduction to Cloud Computing

Kubernetes

Emiliano Casalicchio, Phd

Department of Computer Science and Engineering

Blekinge Institute of Technology

Agenda

References:
https://kubernetes.io/docs/home/
https://kubernetes.io/docs/concepts/

- Kubernetes
  - Architecture overview
  - Main concepts
    - Objects
    - Pods
    - Node
    - Service
  - Control plane components
    - Scheduler
  - Node components

Container Orchestration

- Kubernetes (K8s) is an open source **container orchestration** engine for automating deployment, scaling, and management of containerized applications
- Container Orchestration vs Service Orchestration
  - In service orchestration a central entity coordinate the execution of services. The orchestrator executes a workflow describing the business logic of the application
  - In container orchestration K8s centrally controls and works for
    - The deployment of containerized application
    - The scaling of containers (or K8s nodes)
    - The management of containerized applications (that cover many aspects)

  Deployment, scaling, and management require the execution of a set of actions (think each of them as described by a workflow) and K8s takes care of execution all these worflows.

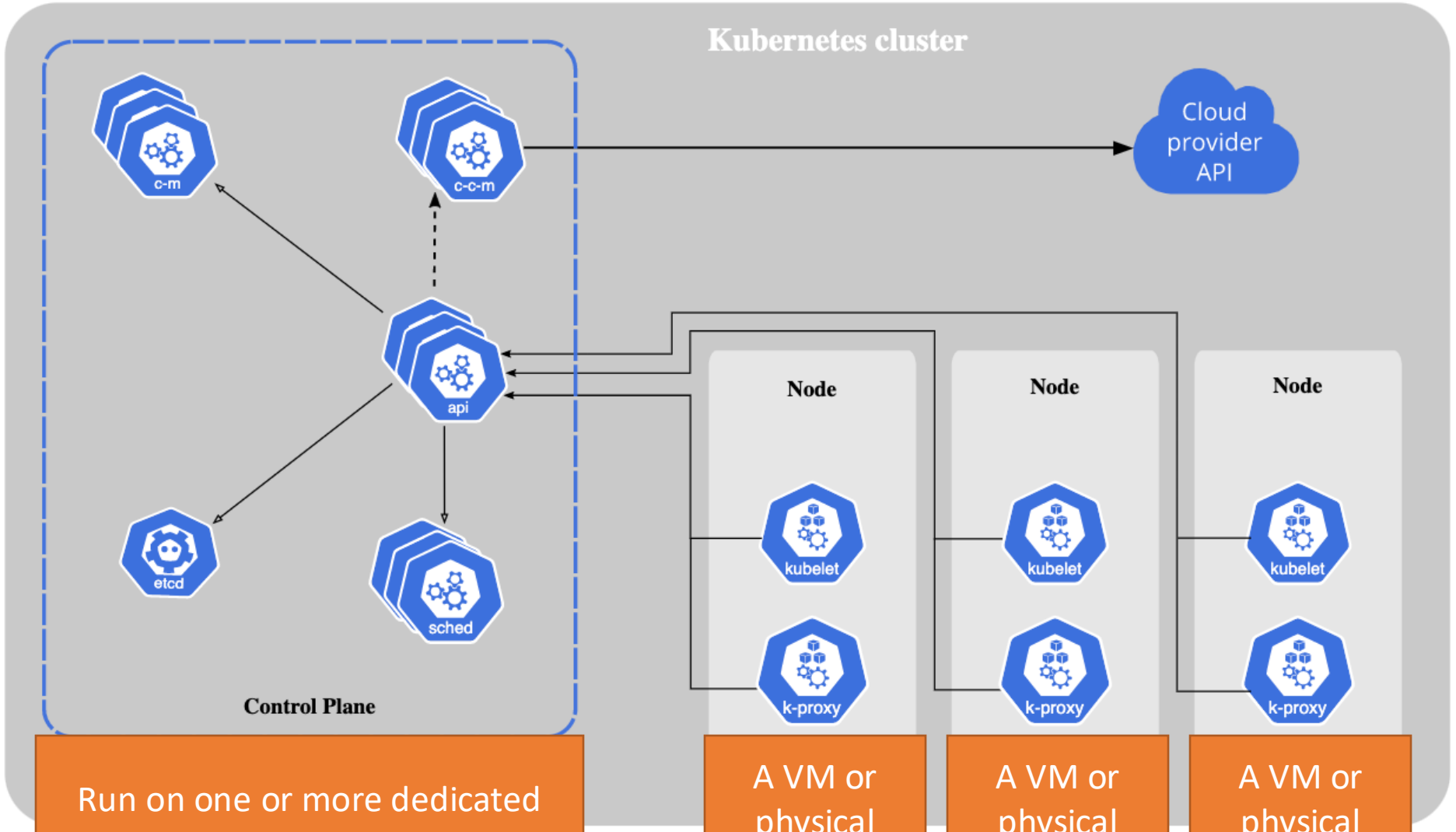
Managing containerized workload

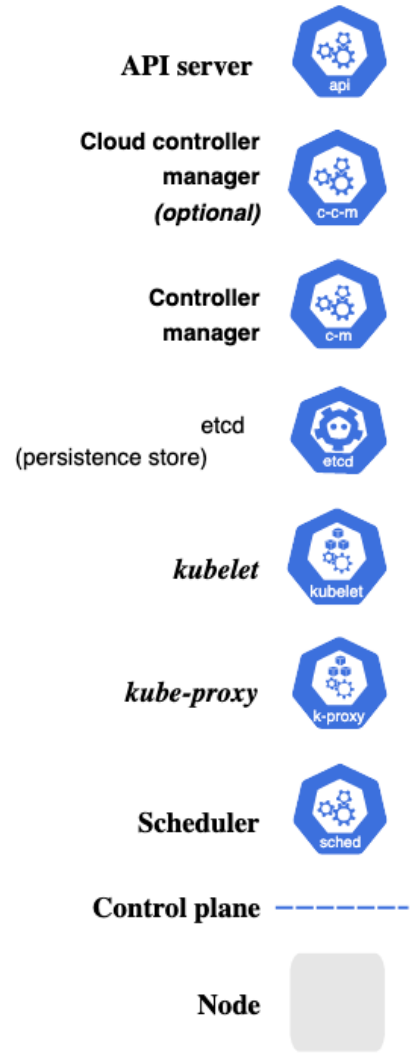
- **Service discovery and load balancing**
- **Storage orchestration;** automatically mount storage
- **Automated rollouts and rollbacks;** automatically bring the deployed containers in a desired (described) state
- **Automatic bin packing (resource scheduling);** fit containers onto cluster nodes to make best use of resources; assign the needed resources (CPU and Memory) to containers
- **Self-healing;** restart, replace, kill, don’t advertise faulty containers
- **Secret and configuration management;**
- **Horizontal scaling**
- **IPv4/IPv6 dual stack**

Kubernetes curiosities!!

- Kubernetes originates from Greek, meaning helmsman or pilot
- 8 in K8s means the eight letters between K and s
- Kubernetes released open-source by google in 2014, project managed now by CNCF
- In 2023, 66% of potential/actual consumers were using Kubernetes in production and 18% were evaluating it (84% total)
  - CNCF annual report https://www.cncf.io/reports/cncf-annual-survey-2023/

K8s cluster





Run on one or more dedicated servers or VMs

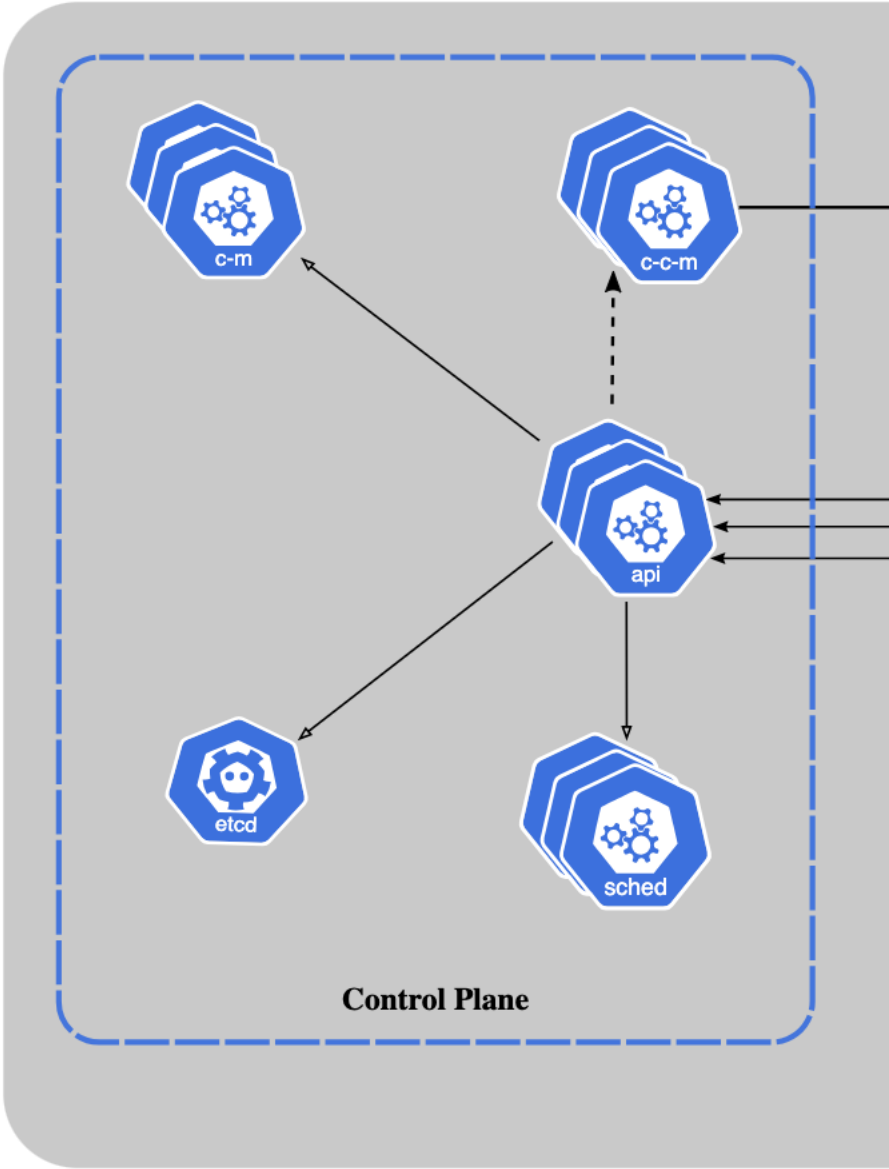
A VM or physical server

A VM or physical server

A VM or physical server

Control Plane components

- Manage the overall state of the cluster
  - kube-apiserver
    - The core component server that exposes the Kubernetes HTTP API
  - etcd
    - Consistent and highly-available key value store for all API server data
  - kube-scheduler
    - Looks for Pods not yet bound to a node, and assigns each Pod to a suitable node
  - kube-controller-manager
    - Runs controllers to implement Kubernetes API behavior
  - cloud-controller-manager (optional)
    - Integrates with underlying cloud provider(s).



Node components

- Run on every node, maintaining running pods and providing the Kubernetes runtime environment
  - Kubelet
    - Ensures that **Pods** are running, including their containers
  - kube-proxy
    - Maintains network rules on nodes to implement **Services**
    - **I**n K8s, a Service is a method for exposing a network application that is running as one or more Pods in your cluster
  - Container runtime
    - Software responsible for running containers
    - K8s requires to use a runtime that conforms with the Container Runtime Interface (CRI)
    - Examples: containerd, CRI-O, Docker Engine, Mirantis Container Runtime

- We need three concepts
  - Objects
  - Pods
  - Service
  - Nodes/Node status

K8s Objects

- *K8s objects* are persistent entities in K8s and are used to represent the state of the cluster. Specifically, they can describe:
  - What containerized applications are running (and on which nodes)
  - The resources available to those applications
  - The policies around how those applications behave, such as restart policies, upgrades, and fault-tolerance
- A K8s object is a "record of intent"--once the object is created, the K8s system will constantly work to ensure that object exists.

K8s Objects: spec & status

- An object contains two fields:
  - spec – describe the **desired state**
    - E.g. 3 instances of the application
  - status – describe the **current state** of the object
    - E.g. 2 instances are currently up&running

Example

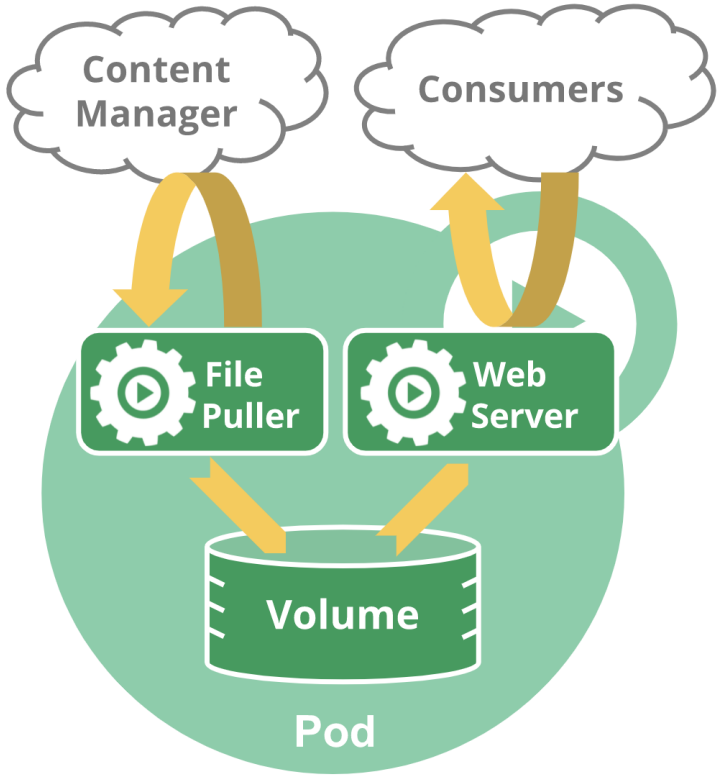
- A deployment object which represent an application running on your cluster
- The object is
  - describe by a manifesto (YAML file)
  - Created invoking an API of the kube-control-manager e.g. through the CLI

  kubectl apply –f <filename>

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2 # tells deployment to run 2 pods matching the template
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Pods

- The smallest deployable units of computing that can be created and managed in K8s.
- Models an application-specific "logical host": it contains one or more application containers which are relatively tightly coupled, and share
  - Storage resources
  - Network resources
  - A specification for how to run the containers.
- A Pod's contents are always co-located and co-scheduled, and run in a shared context.
- The shared context of a Pod is a set of Linux namespaces, cgroups, and potentially other facets of isolation - the same things that isolate a Docker container. Within a Pod's context, the individual applications may have further sub-isolations applied.



Use of Pods

- Pods that run a single container
  - This is the most common Kubernetes use case
- Pods that run multiple containers that need to work together
  - Grouping multiple co-located and co-managed containers in a single Pod is a relatively advanced use case. You should use this pattern only in specific instances in which your containers are tightly coupled.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

kubectl apply –f <filename>

Pods and Workload resources

- Typically a pod is not created directly but through a workload resource: Deployment or Job (eventually a StatefulSet resource)
- In this way K8s take care about workload management actions such as scheduling and scheling to achive the desired state
- When a Pod gets created (directly, or indirectly by a controller),
  - the new Pod is scheduled to run on a Node in the cluster.
  - The Pod remains on that node until
    - the Pod finishes execution,
    - the Pod object is deleted,
    - the Pod is evicted for lack of resources,
    - the node fails.

Pod’s lifecycle

- Under workload

Service

- In Kubernetes, a Service is a method for exposing a network application that is running as one or more Pods in your cluster so that clients can interact with if
  - Pods are ephemeral resources (not expect that an individual Pod is reliable and durable)
  - For a given Deployment in the cluster, the set of Pods running in one moment in time could be different from the set of Pods running that application a moment later
- Problem:
  - if some set of Pods (call them "backends") provides functionality to other Pods (call them "frontends") inside a cluster, how do the frontends find out and keep track of which IP address to connect to, so that the frontend can use the backend part of the workload?
- Answer: Services

Defining a Service

- A Service is an object that can be created, viewed or modified using the Kubernetes API

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Manifest describing a set of Pods that each listen on TCP port 9376 and are labelled as app.kubernetes.io/name=MyApp

Kubernetes assigns this Service an IP address, that is used by the virtual IP address mechanism

The controller for that Service continuously scans for Pods that match its selector, and then makes any necessary updates to the set of EndpointSlices for the Service.

Pod def

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app.kubernetes.io/name: proxy
spec:
  containers:
  - name: nginx
    image: nginx:stable
    ports:
      - containerPort: 80
        name: http-web-svc
```

Service def

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app.kubernetes.io/name: proxy
  ports:
  - name: name-of-service-port
    protocol: TCP
    port: 80
    targetPort: http-web-svc
```

K8s Nodes

- A virtual or physical machine, member of a K8s cluster
- Each node is managed by the control plane and contains the services necessary to run Pods
- Nodes run containerized workload
- Nodes became member of a K8s cluster in two way
  - The kubelet on a node self-registers to the control plane (API-server)
  - A human user manually adds a Node object to the API-server

K8s Nodes

- Once a Node object is created the control plane checks whether the new Node object is valid (all the service are running). If the node object is valid it could run a Pod
- Steps are:
  - K8s creates a Node object internally (the representation).
  - K8s checks that a kubelet has registered to the API server that matches the metadata.name field of the Node.
  - If the node is healthy (i.e. all necessary services are running), then it is eligible to run a Pod. Otherwise, that node is ignored for any cluster activity until it becomes healthy.

JSON manifest describing a node to be created

```
{
  "kind": "Node",
  "apiVersion": "v1",
  "metadata": {
    "name": "10.240.79.157",
    "labels": {
      "name": "my-first-k8s-node"
    }
  }
}
```

K8s node status

- Address
  - hostname, Internal/External IP
- Condition
  - Ready, DiskPressure, MemoryPressure, PidPressure, NetworkUnavailable
- Capacity and Allocatable
  - the resources available on the node: CPU, memory, and the maximum number of pods that can be scheduled onto the node
- Info
  - general information about the node: kernel version, Kubernetes version (kubelet and kube-proxy version), container runtime details, and which operating system the node uses
- Heartbeats (from the K8s nodes to the Kube-controller)
  - updates to the .status of a Node
  - Lease objects (lightweight, reduce the performance impact of updates in large clusters)

Conditions

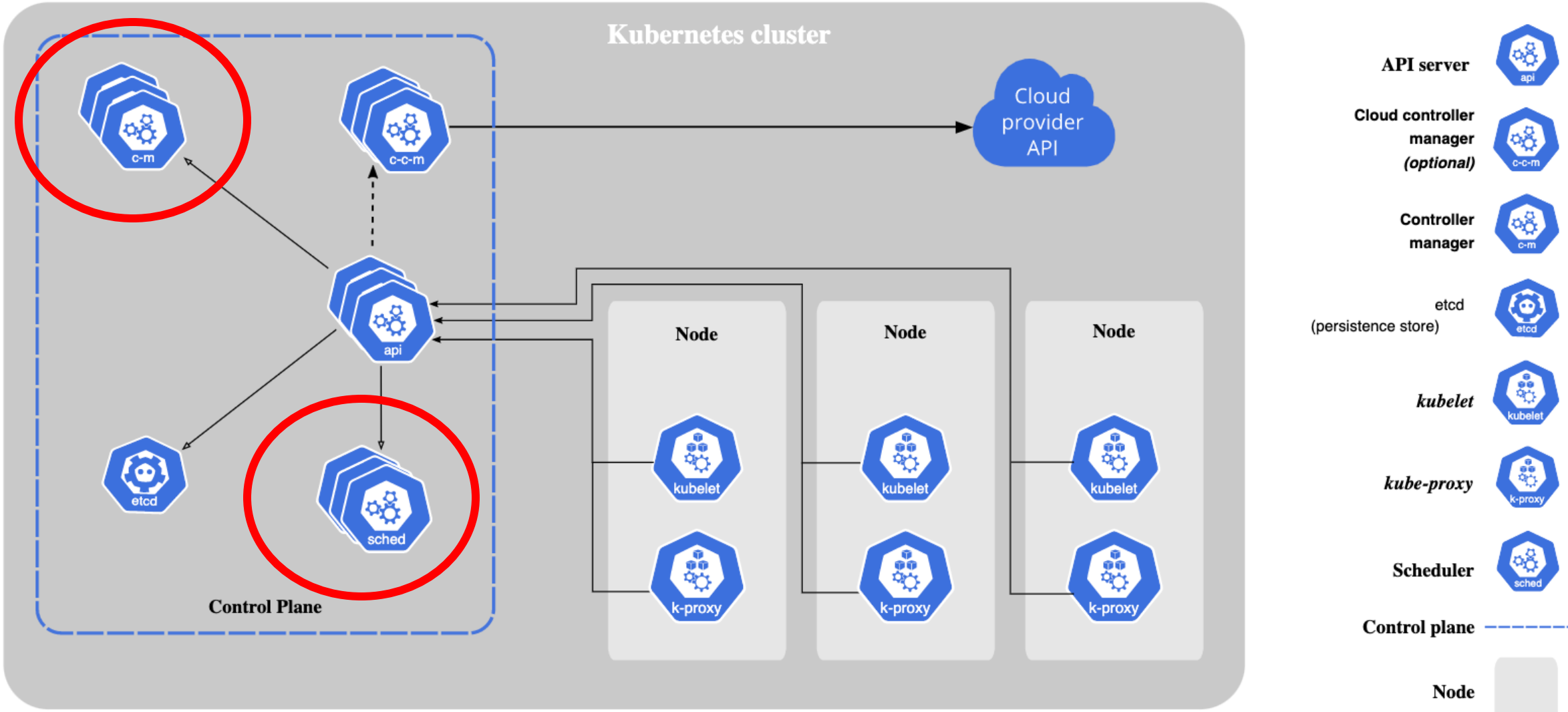
- Ready
  - True if the node is healthy and ready to accept pods, False if the node is not healthy and is not accepting pods, and Unknown if the node controller has not heard from the node in the last node-monitor-grace-period (default is 40 seconds)
- DiskPressure
  - True if pressure exists on the disk size; otherwise False
- MemoryPressure
  - True if pressure exists on the node memory; otherwise False
- PIDPressure
  - True if pressure exists on the processes—that is, if there are too many processes on the node; otherwise False
- NetworkUnavailable
  - True if the network for the node is not correctly configured, otherwise False

JSON structure describing an healty node

```
"conditions": [
  {
    "type": "Ready",
    "status": "True",
    "reason": "KubeletReady",
    "message": "kubelet is posting ready status",
    "lastHeartbeatTime": "2019-06-05T18:38:35Z",
    "lastTransitionTime": "2019-06-05T11:41:27Z"
  }
]
```

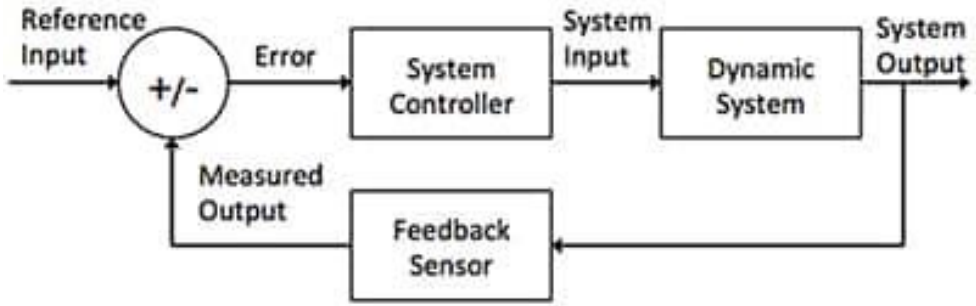
If status is unknown or false the node controller triggers API-initiated eviction for all Pods assigned to that node.
The default eviction timeout duration is 5 minutes

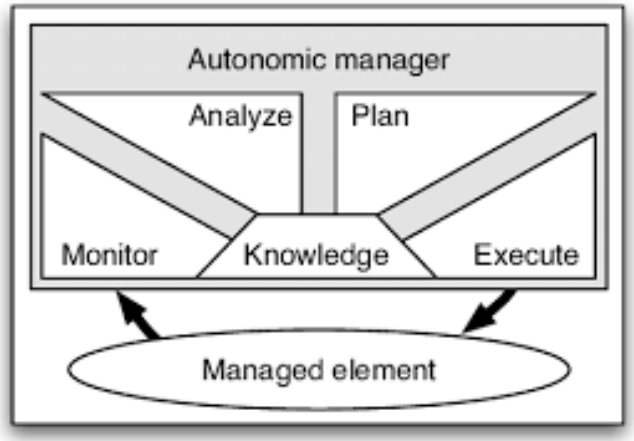
- Now we jump back to the control plane



K8s Controllers

- Controllers are control loops that watch the state of a K8s cluster, then make or request changes where needed.
- A controller tracks at least one Kubernetes resource type.
  These objects have a spec field that represents the desired state. The controller(s) for that resource are responsible for making the current state come closer to that desired state.
- A controller will send messages to the API server that have useful side effects





K8s controllers

- kube-controller-manager
  - Runs controller processes, e.g.
    - Node controller: Responsible for noticing and responding when nodes go down.
    - Job controller: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
    - EndpointSlice controller: Populates EndpointSlice objects (to provide a link between Services and Pods).
    - ServiceAccount controller: Create default ServiceAccounts for new namespaces

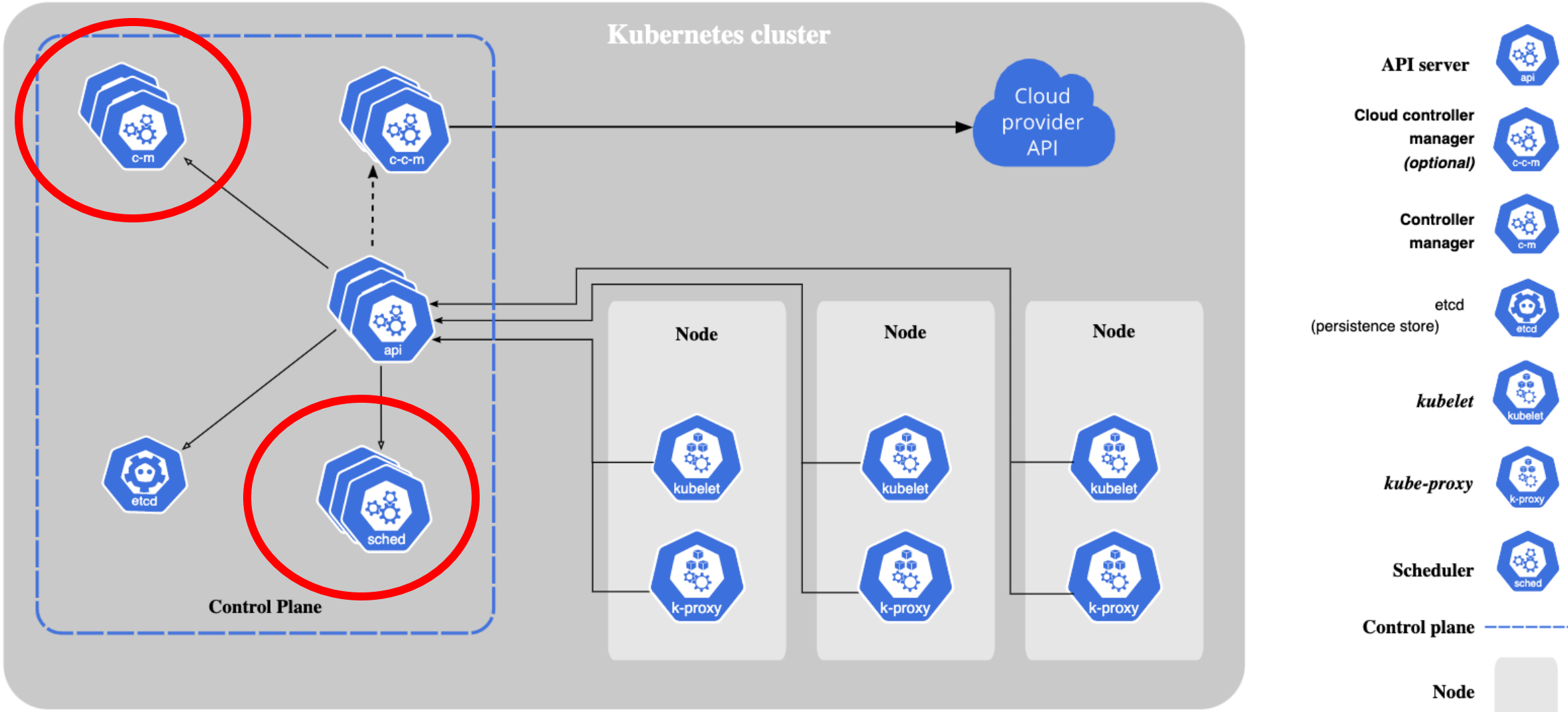
Example: Job controller

- Job is a Kubernetes resource that runs a Pod, or perhaps several Pods, to carry out a task and then stop
  - Once scheduled, Pod objects become part of the desired state for a kubelet
- When the Job controller sees a new task it makes sure that, somewhere in your cluster, the kubelets on a set of Nodes are running the right number of Pods to get the work done
- The Job controller does not run any Pods or containers itself
- The Job controller tells the API server to create or remove Pods
  - Other components in the control plane act on the new information (there are new Pods to schedule and run), and eventually the work is done.
- After a new Job is created, the desired state is for that Job to be completed.
  - The Job controller makes the current state for that Job be nearer to your desired state: creating Pods that do the work you wanted for that Job, so that the Job is closer to completion.
- Once the work is done for a Job, the Job controller updates that Job object to mark it Finished

Example: Control of external cluster resources (Direct control)

- For example: **to make sure there are enough Nodes in your cluster**
- Controllers that interact with external state find their desired state from the API server, then communicate directly with an external system to bring the current state closer in line.
- K8s has an autoscaler controller https://github.com/kubernetes/autoscaler/
  - Cluster autoscaler
    - a component that automatically adjusts the size of a Kubernetes Cluster so that all pods have a place to run and there are no unneeded nodes
  - Vertical pod-autoscaler
    - a set of components that automatically adjust the amount of CPU and memory requested by pods running in the Kubernetes Cluster

- Now we jump back to the control plane



kube-scheduler

- Watches for newly created **Pods** with no assigned node, and selects a node for them to run on
- Since containers in pods - and pods themselves - can have different requirements, the scheduler filters out any nodes that don't meet a Pod's specific scheduling needs
  - In a cluster, Nodes that meet the scheduling requirements for a Pod are called *feasible nodes*.
  - If none of the nodes are suitable, the pod remains unscheduled until the scheduler is able to place it
- scheduling decisions based on:
  - individual and collective resource requirements,
  - hardware/software/policy constraints,
  - affinity and anti-affinity specifications,
  - data locality,
  - inter-workload interference
  - deadlines

Node selection in kube-scheduler

- The scheduler
  - finds feasible Nodes for a Pod (**Filtering**)
  - then runs a set of functions to score the feasible Nodes (**Scoring**)
  - picks a Node with the highest score among the feasible ones to run the Pod.
  - notifies the API server about this decision in a process called **binding**.
- Example of factors that need to be taken into account for scheduling decisions
  - individual and collective resource requirements,
  - hardware / software / policy constraints,
  - data locality,
  - inter-workload interference

Node selection in Kube-scheduler

- Filtering
  - finds the set of Nodes where it's feasible to schedule the Pod
    - PodFitsResources filter checks whether a candidate Node has enough available resource to meet a Pod's specific resource requests
    - Can explore all the cluster’s nodes or only a subset (e.g. 50% of the nodes)
  - Produces the node list containing any suitable Nodes;
    - often, there will be more than one.
    - If the list is empty, that Pod isn't (yet) schedulable
- Scoring
  - The scheduler assigns a score to each Node that survived filtering, basing this score on the active scoring rules.
  - Finally, kube-scheduler assigns the Pod to the Node with the highest ranking.
    - If there is more than one node with equal scores, kube-scheduler selects one of these at random.

Configuration of Filtering and Scoring

- Scheduling Policies (not supported since Kubernetes v1.23)
  - allow you to configure
    - Predicates for filtering
    - Priorities for scoring.
- Scheduling Profiles
  - allow you to configure Plugins that implement different scheduling stages, including:
    - QueueSort, Filter, Score, Bind, Reserve, Permit, and others
  - Used to customize kube-scheduler

Scheduling policies (Example of predicates)

- PodFitsHostPorts:
  - Checks if a Node has free ports (the network protocol kind) for the Pod ports the Pod is requesting.
- PodFitsHost:
  - Checks if a Pod specifies a specific Node by its hostname.
- PodFitsResources:
  - Checks if the Node has free resources (eg, CPU and Memory) to meet the requirement of the Pod.
- NoVolumeZoneConflict:
  - Evaluate if the Volumes that a Pod requests are available on the Node, given the failure zone restrictions for that storage.
- NoDiskConflict:
  - Evaluates if a Pod can fit on a Node due to the volumes it requests, and those that are already mounted.

Scheduling policies (Example of predicates)

- MatchNodeSelector:
  - Checks if a Pod's Node Selector matches the Node's label(s).
- MaxCSIVolumeCount:
  - Decides how many Container Storage Interface volumes should be attached, and whether that's over a configured limit.
- CheckVolumeBinding:
  - Evaluates if a Pod can fit due to the volumes it requests.

Scheduling policies (Example of Priorities)

- SelectorSpreadPriority:
  - Spreads Pods across hosts, considering Pods that belong to the same Service, StatefulSet or ReplicaSet.
- LeastRequestedPriority:
  - Favors nodes with fewer requested resources. In other words, the more Pods that are placed on a Node, and the more resources those Pods use, the lower the ranking this policy will give.
- MostRequestedPriority:
  - Favors nodes with most requested resources. This policy will fit the scheduled Pods onto the smallest number of Nodes needed to run your overall set of workloads.
- BalancedResourceAllocation:
  - Favors nodes with balanced resource usage.

Scheduling policies (Example of Priorities)

- NodePreferAvoidPodsPriority:
  - Prioritizes nodes according to the node annotation. You can use this to hint that two different Pods shouldn't run on the same Node.
- ImageLocalityPriority:
  - Favors nodes that already have the container images for that Pod cached locally.
- ServiceSpreadingPriority:
  - For a given Service, this policy aims to make sure that the Pods for the Service run on different nodes. It favours scheduling onto nodes that don't have Pods for the service already assigned there. The overall outcome is that the Service becomes more resilient to a single Node failure.
- EqualPriority:
  - Gives an equal weight of one to all nodes.

Scheduling profiles

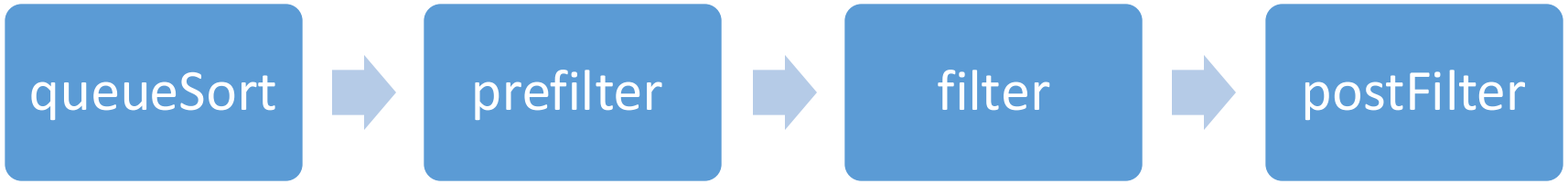
- A scheduling Profile allows to configure the different stages of scheduling in the kube-scheduler.
- Each stage is exposed in an **extension point**
  - Plugins provide scheduling behaviors by implementing one or more of these extension points

```yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - plugins:
      score:
        disabled:
        - name: PodTopologySpread
        enabled:
        - name: MyCustomPluginA
          weight: 2
        - name: MyCustomPluginB
          weight: 1
```

Extension point

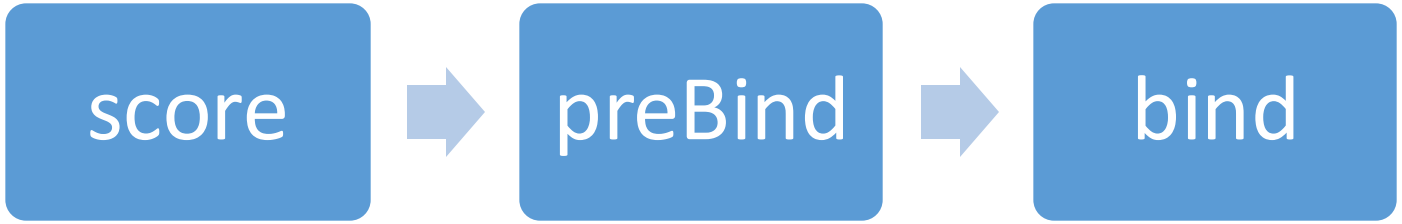
Plogins implementing the extension point

Extantion points (sequence for filtering)



- **queueSort**: These plugins provide an ordering function that is used to sort pending Pods in the scheduling queue. Exactly one queue sort plugin may be enabled at a time.
- **preFilter**: These plugins are used to pre-process or check information about a Pod or the cluster before filtering. They can mark a pod as unschedulable.
- **filter**: These plugins are the equivalent of Predicates in a scheduling Policy and are used to filter out nodes that can not run the Pod. Filters are called in the configured order. A pod is marked as unschedulable if no nodes pass all the filters.
- **postFilter**: These plugins are called in their configured order when no feasible nodes were found for the pod. If any postFilter plugin marks the Pod schedulable, the remaining plugins are not called.

Extantion points (sequence for scoring and binding)

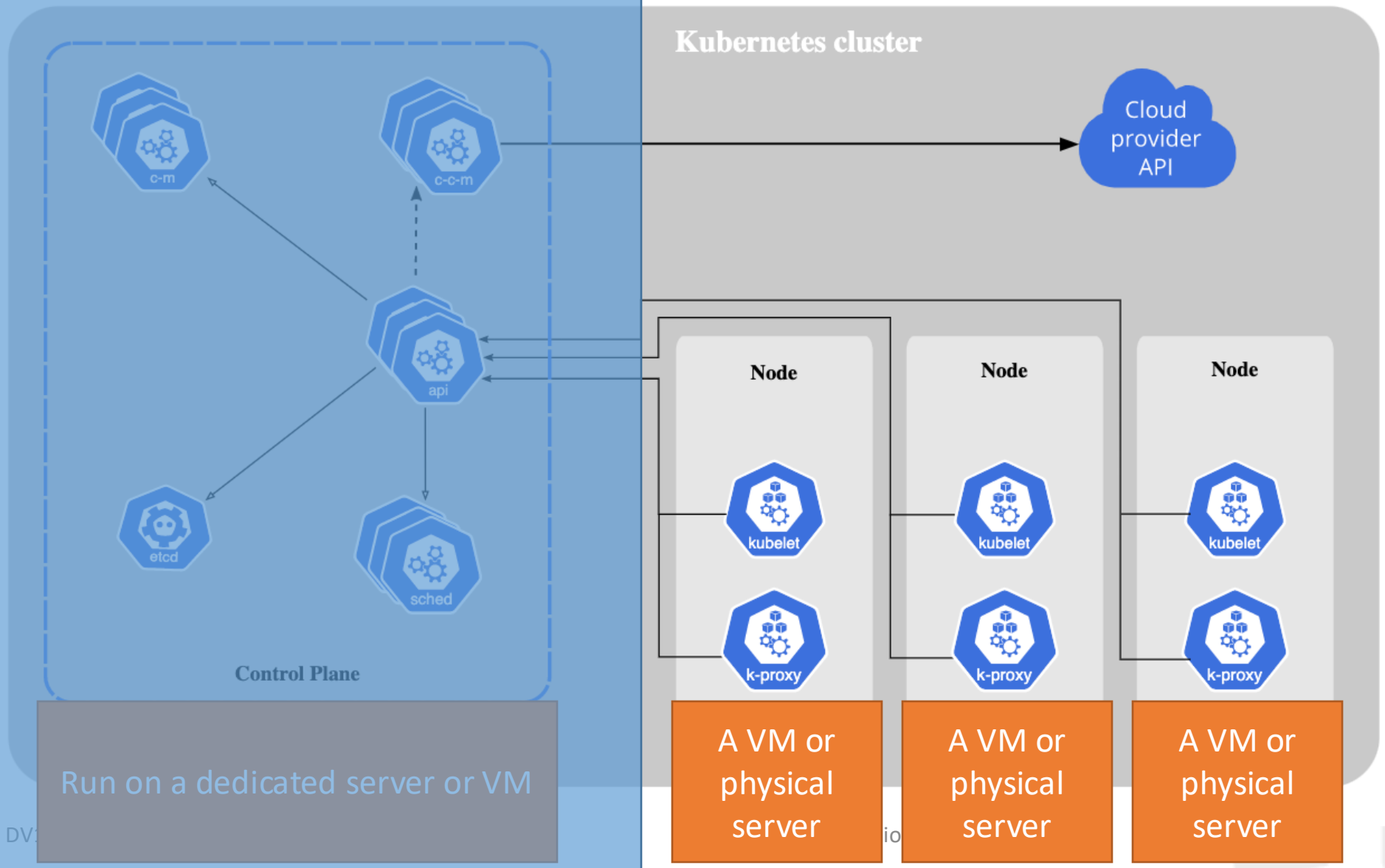


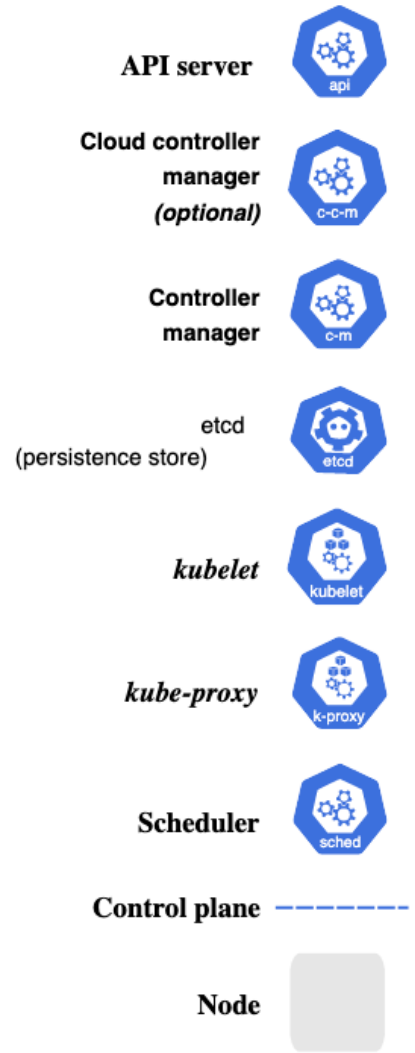
- **score:** These plugins provide a score to each node that has passed the filtering phase. The scheduler will then select the node with the highest weighted scores sum.
- **preBind:** These plugins perform any work required before a Pod is bound.
- **bind:** The plugins bind a Pod to a Node. bind plugins are called in order and once one has done the binding, the remaining plugins are skipped. At least one bind plugin is required

Plugins: extention point implementation

- Filter
  - NodeResourcesFit: Checks if the node has all the resources that the Pod is requesting. The score can use one of three strategies: LeastAllocated (default), MostAllocated and RequestedToCapacityRatio
  - VolumeBinding: Checks if the node has or if it can bind the requested volumes
  - VolumeRestrictions: Checks that volumes mounted in the node satisfy restrictions that are specific to the volume provider
- Score
  - ImageLocality: Favors nodes that already have the container images that the Pod runs
  - NodeResourcesFit
  - NodeResourcesBalancedAllocation: Favors nodes that would obtain a more balanced resource usage if the Pod is scheduled there

K8s cluster


Kubernetes cluster
Cloud provider API
c-m
c-c-m
api
etcd
sched
Control Plane
Node
Node
Node
kubelet
kubelet
kubelet
k-proxy
k-proxy
k-proxy
Run on a dedicated server or VM
A VM or physical server
A VM or physical server
A VM or physical server


API server
Cloud controller manager (optional)
Controller manager
etcd (persistence store)
kubelet
kube-proxy
Scheduler
Control plane
Node